



PROFEVIXO • GUÍA DE ESTUDIO

Teoría de la computabilidad

Máquinas de Turing y decidibilidad.

Matemática discreta y fundamentos

Univ 4-5

20 EJERCICIOS

TEORÍA + RESULTADOS

Teoría de la computabilidad

Máquinas de Turing y decidibilidad.

1. Máquinas de Turing

Una **máquina de Turing (MT)** es un modelo abstracto de cómputo formado por una cinta infinita dividida en celdas, un cabezal que lee y escribe, y un conjunto finito de estados con reglas de transición.

Definición (formal). Una MT es una tupla

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{aceptar}}, q_{\text{rechazar}})$$

donde Q es el conjunto finito de estados, Σ el alfabeto de entrada, Γ el alfabeto de cinta con $\Sigma \subseteq \Gamma$, $\sqcup \in \Gamma \setminus \Sigma$ el símbolo blanco, q_0 el estado inicial, q_{aceptar} y q_{rechazar} los estados de aceptación y de rechazo, y

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

la función de transición en la versión determinista.

Funcionamiento. En cada paso la MT lee el símbolo bajo el cabezal y, según δ , escribe un símbolo, mueve el cabezal a la izquierda o a la derecha y cambia de estado. La máquina se detiene al alcanzar q_{aceptar} o q_{rechazar} .

Ejemplo. Una máquina que suma 1 a un número binario: recorre la cinta de derecha a izquierda cambiando cada 1 por 0 hasta encontrar el primer 0, que cambia por 1.

Variantes. Las cintas múltiples, la cinta bidimensional y las máquinas no deterministas resultan equivalentes en poder de cómputo.

2. Tesis de Church-Turing y modelos equivalentes

Tesis de Church-Turing. Todo lo que es efectivamente computable es computable por una máquina de Turing.

No es un teorema demostrable, sino una tesis que identifica la noción intuitiva de "algoritmo" con el cómputo de Turing; no ha sido refutada y sostiene toda la teoría.

Modelos equivalentes. Tienen el mismo poder que las MT:

1 El **cálculo lambda** de Church.

2 Las **funciones recursivas** (composición, recursión primitiva y minimización).

3 Las **máquinas de registros** y los lenguajes de programación razonables (Python, C, etc.) con memoria ilimitada.

4 Las **gramáticas formales** de tipo 0, sin restricciones.

Ejemplo. La función $f(n) = n + 1$ y el algoritmo de Euclides son Turing-computables; cualquier programa de alto nivel puede simularse con una MT.

Consecuencia. Los resultados de imposibilidad, como la indecidibilidad de la parada, valen para todo modelo de cómputo.

3. Decidibilidad y lenguajes

Un **lenguaje** es un conjunto de cadenas $L \subseteq \Sigma^*$. Los problemas de decisión ($\{w \in L?\}$) se clasifican según la máquina que los resuelve:

1 **Decidible (recursivo).** Existe una MT que siempre termina y responde correctamente "sí" o "no" para toda entrada.

2 **Reconocible (recursivamente enumerable, r.e.).** Existe una MT que acepta toda cadena de L y puede no terminar o rechazar con las que no están en L .

3 **Indecidible.** Ninguna MT lo decide.

Teorema. Todo lenguaje decidible es reconocible.

Teorema. Existe un lenguaje reconocible que no es decidible: el problema de la parada.

Ejemplo. Los lenguajes regulares y los libres de contexto son decidibles; en cambio, $\{0^n 1^n : n \geq 0\}$ es decidible pero no regular.

Cardinalidad. Hay más lenguajes que máquinas, pues los lenguajes forman un conjunto no numerable y las MT uno numerable; por eso existen lenguajes que ni siquiera son reconocibles.

4. El problema de la parada

Definición. El problema de la parada es el lenguaje

$$H = \{\langle M, w \rangle : M \text{ se detiene con la entrada } w\}.$$

Teorema (Turing, 1936). H es indecidible.

Demostración. Supongamos que existe una MT D que decide H . Construimos una MT P que, con entrada $\langle M \rangle$, simula D sobre $\langle M, \langle M \rangle \rangle$ y hace lo contrario: si D responde "se detiene", P entra en un bucle infinito; si D responde "no se detiene", P acepta. Al evaluar P con su propia descripción, $P(\langle P \rangle)$ se detiene si y solo si no se detiene, una contradicción. Por lo tanto D no existe. ■

Consecuencia. No existe un verificador general de terminación ni un detector perfecto de bucles infinitos.

Ejemplo. Aunque la parada sea indecidible en general, para el programa concreto "mientras verdadero: nada" la respuesta es inmediata.

5. Reducciones

Una **reducción de A a B** es una transformación computable f de instancias de A en instancias de B tal que

$$x \in A \iff f(x) \in B.$$

Se escribe $A \leq_m B$ (reducción de muchos a uno).

Teorema. Si $A \leq_m B$ y A es indecidible, entonces B es indecidible.

Demostración. Si B fuese decidable mediante M_B , entonces $M_B \circ f$ decidiría A , una contradicción. ■

Método estándar. Para probar que un problema B es indecidible se reduce la parada, o su complemento, a B ; nunca al revés.

Ejemplos de problemas indecidibles por reducción:

1 ¿Se detiene M con la entrada vacía?

2 ¿Acepta M alguna cadena?

3 ¿Son equivalentes dos máquinas, es decir, tienen el mismo lenguaje?

4 El **problema de correspondencia de Post**: dadas parejas de cadenas, ¿existe una secuencia que iguale la parte superior y la inferior?

Ejemplo de reducción. Para reducir " M acepta ε ?" a la parada se construye M' que borra su entrada y simula M sobre ε . Entonces M' se detiene si y solo si M acepta ε .

6. El teorema de Rice

Teorema de Rice. Toda propiedad **semántica no trivial** del lenguaje de una máquina de Turing es indecidible.

Una propiedad de lenguajes P es:

1 **Semántica:** depende solo del lenguaje $L(M)$ y no de la forma de la máquina M .

2 **No trivial:** la cumple algún lenguaje reconocible y no la cumple otro.

Ejemplos indecidibles. "El lenguaje es finito", "el lenguaje es vacío", "la máquina acepta 0", "el lenguaje es regular", "el lenguaje es igual a Σ^* ".

Lo que Rice no dice. No prohíbe decidir propiedades **sintácticas**, como "tiene más de 100 estados", ni propiedades de máquinas particulares. La frontera es semántica frente a sintáctica.

Ejemplo. " $L(M) = \emptyset$?" es semántica y no trivial: por Rice es indecidible. En cambio, "¿tiene M menos de 10 estados?" es sintáctica y se decide contando estados.

7. Recursivo frente a recursivamente enumerable

Recordemos las dos clases:

- 1 **Recursivo (decidible):** el algoritmo termina en todos los casos.
- 2 **r.e. (reconocible):** el algoritmo acepta las cadenas del lenguaje, pero puede no terminar.

Teorema de Post. Un lenguaje L es decidible si y solo si L y su complemento $\overline{L}$ son reconocibles (r.e.).

Idea. Si L y $\overline{L}$ son r.e., se ejecutan ambas máquinas en paralelo (*dovetailing*); una de las dos terminará y decide L . El recíproco es inmediato, pues un decisor reconoce L y $\overline{L}$. ■

Consecuencia. El complemento de la parada $\overline{H}$ no es r.e., porque H es r.e. y no decidible.

Jerarquía. Decidibles $\subset$ r.e. $\subset$ todos los lenguajes; las diferencias se afinan con la jerarquía aritmética y los grados de Turing.

Ejemplo. La parada es r.e., pues simulamos la máquina que termina si se detiene, pero no es decidible.

8. Límites de la computabilidad

Tesis de Church-Turing fuerte (física). Cualquier función computable por un proceso físico razonable es Turing-computable; no se conoce hipercomputación realizable.

Límites de la matemática automatizada:

- 1 No existe un demostrador automático completo para toda la matemática, ligado a la indecidibilidad y a la incompletitud.
- 2 La verificación de software debe usar lógica restringida o asistencia humana.
- 3 Los asistentes de pruebas, como Coq o Lean, verifican demostraciones dadas, no las inventan.

Lado positivo. La indecidibilidad separa lo posible de lo imposible y orienta la investigación hacia problemas que sí pueden automatizarse, como el análisis estático con anotaciones o los dominios restringidos.

Ejemplo. Un compilador puede advertir de bucles evidentes, pero un detector perfecto de bucles infinitos sería un decisor de la parada: es imposible.

Ejercicios (20)

Nivel 1 · Básico

- 1 Define formalmente una máquina de Turing e indica qué representa cada componente de la tupla.
- 2 Enuncia la tesis de Church-Turing.

3 ¿Qué significa que un lenguaje sea decidable?

4 ¿Qué diferencia hay entre un lenguaje reconocible y uno decidable?

5 ¿Es decidable el problema de la parada? Justifica en una frase.

Nivel 2 · Intermedio

6 Describe informalmente una MT que reconoce el lenguaje $\{0^n 1^n : n \geq 0\}$.

7 Explica por qué la tesis de Church-Turing no es un teorema demostrable.

8 Da un ejemplo de lenguaje no decidable pero reconocible.

9 ¿Por qué existen lenguajes que no son reconocibles? Usa un argumento de cardinalidad.

10 Enuncia el teorema de Rice y da dos propiedades cubiertas por él.

Nivel 3 · Avanzado

11 Demuestra que si $A \leq_m B$ y A es indecidible, entonces B es indecidible.

12 Reduce el problema " M acepta ε ?" al problema de la parada.

13 Explica por qué el complemento del problema de la parada no es r.e.

14 Aplica el teorema de Rice para decidir la decidibilidad de " $L(M)$ es finito?".

15 ¿Es decidable " M tiene más de 5 estados?". Justifica.

Nivel 4 · Desafío

16 Describe la demostración diagonal de la indecidibilidad del problema de la parada.

17 Explica por qué, por el teorema de Post, decidir un lenguaje equivale a reconocerlo junto con su complemento.

18 Reduce el problema de la parada al problema " M se detiene con la entrada vacía?".

19 ¿Por qué un detector perfecto de bucles infinitos en un compilador es imposible?

- 20 Explica por qué la existencia de lenguajes no r.e. no contradice que los problemas prácticos sean decidibles.

Respuestas

- 1 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{aceptar}, q_{rechazar})$: estados, alfabeto de entrada, alfabeto de cinta, transición, estado inicial y estados de aceptación y rechazo.
- 2 Todo lo efectivamente computable es computable por una máquina de Turing.
- 3 Que existe una MT que siempre termina y responde correctamente "sí" o "no" para toda entrada.
- 4 Un reconocedor puede no terminar con las cadenas que no pertenecen al lenguaje; un decisor siempre termina.
- 5 No: el teorema de Turing prueba que es indecidible mediante un argumento diagonal.
- 6 Marca alternadamente un 0 y un 1; si ambos se agotan a la vez, acepta.
- 7 Porque identifica una noción informal, la de "procedimiento efectivo", con un modelo formal; no es enunciable como teorema dentro de un sistema.
- 8 El problema de la parada H : es r.e. pero no decidable.
- 9 Hay una cantidad no numerable de lenguajes y solo una numerable de máquinas, luego sobran lenguajes sin máquina que los reconozca.
- 10 Toda propiedad semántica no trivial de $L(M)$ es indecidible; por ejemplo "el lenguaje es finito" y "el lenguaje es vacío".
- 11 Si B fuese decidable por M_B , la composición $M_B \circ f$ decidiría A , contradiciendo su indecidibilidad.
- 12 Construye M' que borra la entrada y simula M sobre ε ; M' se detiene si y solo si M acepta ε .
- 13 Si $\overline{H}$ fuese r.e., como H también lo es, por el teorema de Post H sería decidable, lo cual es falso.
- 14 "Ser finito" es una propiedad semántica no trivial, luego es indecidible por Rice.
- 15 Sí es decidable: es una propiedad sintáctica que se comprueba contando los estados de la máquina.

- 16 Se supone un decisor D de la parada y se construye P que hace lo contrario de lo que D responde sobre $\langle M, \langle M \rangle \rangle$; con $\langle P \rangle$ se obtiene una contradicción.
- 17 Si L y $\overline{L}$ son r.e. se simulan ambas máquinas en paralelo y una termina, decidiendo; un decisor reconoce ambos lenguajes.
- 18 Dada $\langle M, w \rangle$ construye M' que ignora su entrada y simula M sobre w ; M' se detiene con ε si y solo si M se detiene con w .
- 19 Sería un decisor del problema de la parada, que es indecidible; solo se detectan patrones simples.
- 20 La no reconocibilidad es una propiedad de la totalidad de los lenguajes; los problemas concretos de interés suelen caer en clases decidibles o se resuelven con restricciones.